PS-027

IPv6 Website and Dependency Readiness Auditor

Finding where IPv6 fails during real resource loading: DNS, TCP, TLS and HTTP

Devang Gangwar
devanggangwar3@gmail.com

AIORI Hackathon 3

October 2026

Problem Description

Clear outline of the problem being addressed.

- A website is **not** IPv6-ready just because its apex domain has an AAAA record.
- Modern pages pull scripts, stylesheets, fonts, images and API calls from many other hostnames, and some of these stay **IPv4-only**.
- On IPv6-only networks, one IPv4-only dependency can mean broken styling, missing fonts, failed scripts or dead API calls.
- Browsers use Happy Eyeballs, so on dual-stack networks a failing IPv6 path is silently retried over IPv4 and the problem stays **invisible**.
- Existing checkers test only the main hostname and rarely say **which stage** failed (DNS, TCP, TLS or HTTP).

Objective

Build a browser-based auditor that loads a page, maps its **full resource dependency tree**, and tests every dependency over IPv6 and IPv4, isolating the **failure stage** for each resource.

Evaluation: 30 selected real-world sites plus a controlled site seeded with IPv4-only and misconfigured IPv6 assets.

Outputs: dependency graph, failure classification, CSV/JSON reports, technical report and demo.

Background

Why an AAAA record on the main domain is not enough

Deployment models

Networks run dual-stack, IPv6-only, or IPv6-only with NAT64/DNS64 translation [3,4]. IPv6-only and IPv6-mostly access networks are growing [9], so IPv4-only hosts matter more each year.

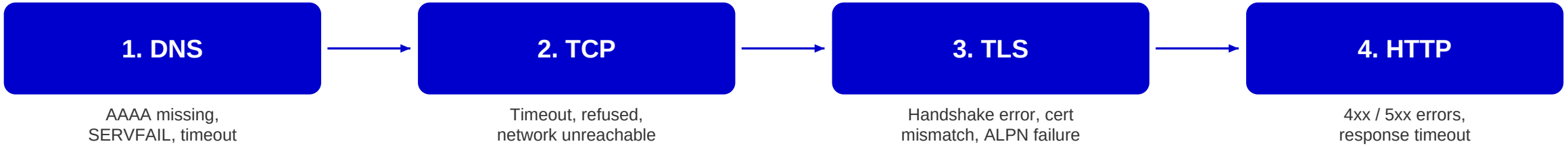
Happy Eyeballs hides faults

Browsers race IPv6 against IPv4 and quickly fall back [1]. Good for users, but it means a broken IPv6 path is masked, and a normal page load cannot prove IPv6 works.

A page is a dependency tree

One HTML page triggers requests to CDNs, font hosts, analytics and APIs, and scripts load other scripts. Every unique hostname needs its own IPv6 check.

Every request passes through four stages. IPv6 can fail at any of them:



Goal of the tool: report the **first stage that fails** for each resource, in both address families, so a failure is traceable to its cause.

Literature & RFC Review

Only the most relevant standards and prior work

Ref	Source	Relevance to this project
[1]	RFC 8305 Happy Eyeballs v2	Explains why dual-stack browsers hide IPv6 failures. Our probes therefore force one address family at a time instead of trusting a normal load.
[2]	RFC 6724 Default Address Selection	Defines how hosts choose between IPv6 and IPv4 destinations; needed to interpret which family a resource used.
[3][4]	RFC 6146 NAT64 and RFC 6147 DNS64	Basis of IPv6-only test networks. Translation can make IPv4-only hosts look reachable, so we compare runs with and without NAT64.
[5]	W3C Resource Timing	What a page can observe about its own loads. It gives timings but not the IP family or stage-level cause, which is why a browser-only page is insufficient.
[6]	Chrome DevTools Protocol (Network domain)	Gives per-request remote IP address, protocol and timing (DNS, connect, SSL), the data source for the capture module.
[7]	IETF Hackathon: webres6	The inspiration for this work: auditing IPv6 reachability of web resources and their dependencies. We extend it with stage classification and a controlled test site.

Proposed Solution

A browser-based auditor backed by a measurement engine

How it works

- **1. Load**: headless Chromium opens the target page and records every request through the DevTools Protocol.
- **2. Map**: each request's initiator builds a dependency tree (page, script, API call; CSS, font).
- **3. Probe**: every unique hostname is tested separately over IPv6 and IPv4: DNS, then TCP, then TLS, then HTTP.
- **4. Classify**: the first failing stage becomes the failure class; IPv4 results act as the control.
- **5. Report**: dashboard graph plus CSV/JSON exports and a per-site readiness score.

Key capabilities

- Browser dashboard: enter a URL or a list, choose a test mode, view results.
- Interactive dependency graph coloured by status: IPv6 OK, degraded, failed, IPv4-only.
- Forced per-family probing, so Happy Eyeballs cannot hide failures.
- Three test modes: dual-stack baseline, IPv6-only, IPv6-only with NAT64/DNS64.
- Optional browser extension for live auditing of the page you are viewing.
- Controlled test site that provides known ground truth for validation.

Failure Classification & Test Site

Per-resource verdict and a controlled environment with known faults

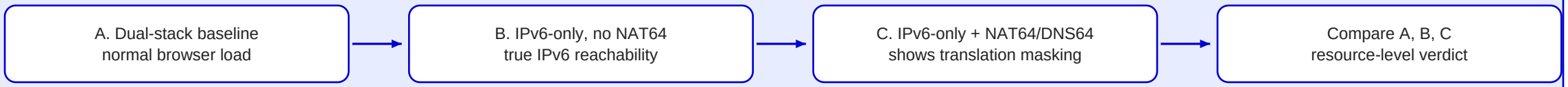
Failure taxonomy (first failing stage)

Stage	How it is tested	Failure classes
DNS	Separate A and AAAA queries	NO_AAAA, AAAA_SERVFAIL, AAAA_TIMEOUT
TCP	Connect to port 443 over an IPv6-only socket	TCP_TIMEOUT, TCP_REFUSED, TCP_UNREACHABLE
TLS	Handshake with SNI over the same socket	TLS_FAIL, CERT_MISMATCH, ALPN_FAIL
HTTP	GET/HEAD over IPv6	HTTP_4XX, HTTP_5XX, HTTP_TIMEOUT
OK	All four stages pass	IPV6_OK (IPv4 run is the control)

Controlled site: one subdomain per seeded fault

Subdomain	Seeded fault	Expected class
ok	A + AAAA, v6 and v4 listening	IPV6_OK
v4only	A record only	NO_AAAA
blackhole	AAAA exists, v6 packets dropped	TCP_TIMEOUT
refused	AAAA exists, v6 port closed	TCP_REFUSED
badtls	v6 vhost serves wrong certificate	CERT_MISMATCH
err500	v6 vhost returns HTTP 500	HTTP_5XX

Test workflow



Ground truth from the test site validates the classifier. Real-world run: 30 sites (about 10 Indian government/education, 10 popular global, 10 third-party-heavy commercial), repeated at different times from a stated IPv6 vantage point.

Optimization Proposed

Before and after comparison

Aspect	Before (typical practice)	After (this tool)
Scope of check	Apex domain has an AAAA record	Every resource in the full dependency tree, including second-level dependencies
Address-family visibility	Happy Eyeballs hides IPv4 fallback	Forced per-family probes plus the real remote IP from the capture
Failure detail	'Page loads' or 'page broken'	Stage-specific class: DNS, TCP, TLS or HTTP, with timings
NAT64 masking	Not detected	Runs with and without NAT64 and flags resources that only work via translation
Validation	Anecdotal	Controlled site with known seeded faults as ground truth
Output	Manual screenshots and notes	CSV/JSON datasets, dependency graph, readiness score per site
Repeatability	Single run	Repeated runs at different times to expose DNS and CDN variation

Targets: classifier agrees with all seeded faults on the test site; each of the 30 sites scanned in dual-stack and IPv6-only modes; every dependency hostname given a verdict.

Solution Architecture & Design

Component view of the auditor

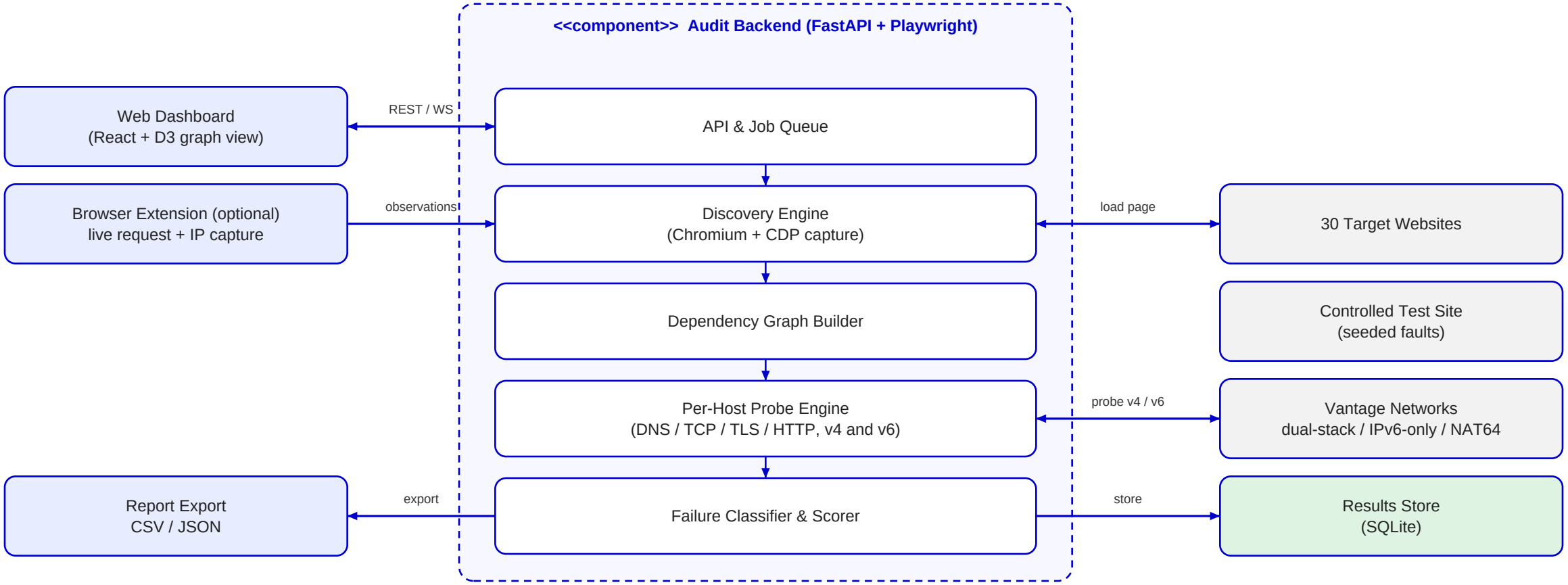


Figure: System architecture (component diagram). Probes target all 30 sites and the controlled test site.

Data Flow Diagram

Overview of how data moves through the auditor

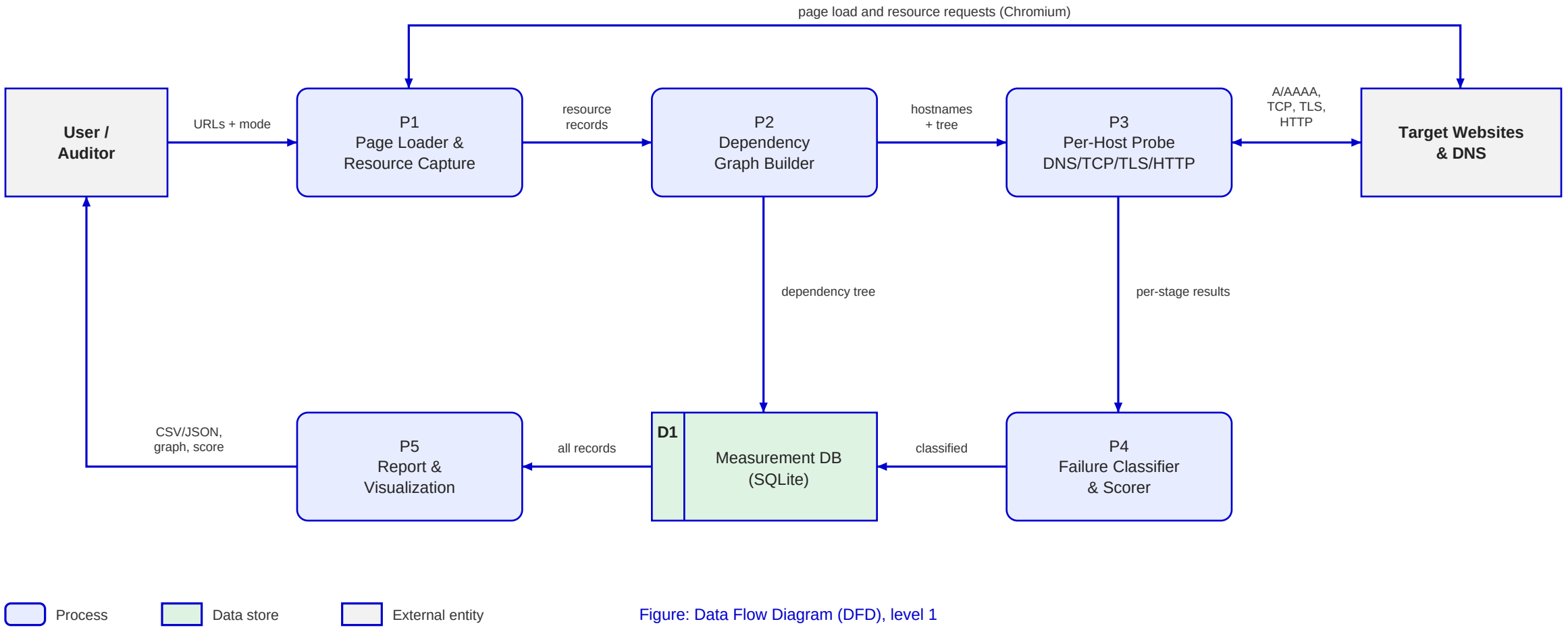
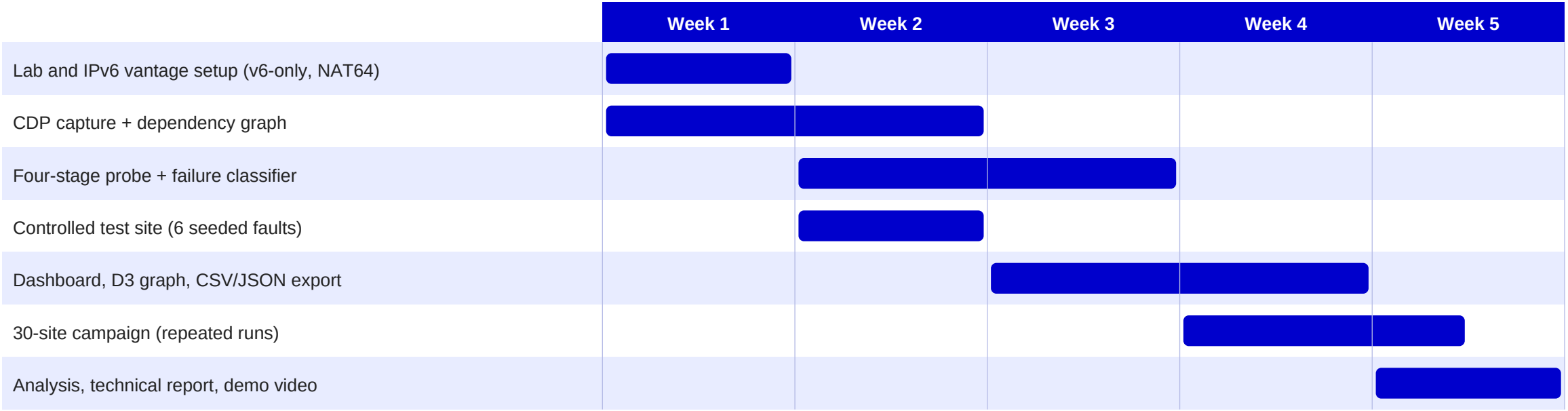


Figure: Data Flow Diagram (DFD), level 1

Deployment Timeline

Milestones and methodology: iterative weekly sprints



Methodology. Build the capture and probe modules first, validate the classifier against the controlled site (known ground truth), then run the 30-site campaign. Each sprint ends with a working increment and exported sample data.

Final deliverables. Browser-based auditor and source code; dependency graph view; controlled IPv4-only test site; DNS/TCP/TLS/HTTP classification; CSV/JSON reports; results for 30 sites; technical report and demonstration.

Bibliography

[1] IETF. RFC 8305: Happy Eyeballs Version 2: Better Connectivity Using Concurrency. December 2017.

[2] IETF. RFC 6724: Default Address Selection for Internet Protocol Version 6 (IPv6). September 2012.

[3] IETF. RFC 6146: Stateful NAT64: Network Address and Protocol Translation from IPv6 Clients to IPv4 Servers. April 2011.

[4] IETF. RFC 6147: DNS64: DNS Extensions for Network Address Translation from IPv6 Clients to IPv4 Servers. April 2011.

[5] W3C. Resource Timing specification (Level 2 / Level 3).

[6] Google. Chrome DevTools Protocol, Network domain documentation.

[7] IETF Hackathon. webres6: IPv6 readiness of web resources and their dependencies.

[8] IETF. RFC 8200: Internet Protocol, Version 6 (IPv6) Specification. July 2017.

[9] IETF. RFC 9386: IPv6 Deployment Status. April 2023.

Thank You for Your Attention!

PS-027: IPv6 Website and Dependency Readiness Auditor

Questions and discussion